

Week 3 — Monday Lab: Users Table & Registration

IM102 – Advanced Database Systems | 3 Hours | Computer Lab

Setup

Copy Week 2 → `im102_lab3`, update config, create `im102_lab3` database, re-run your Week 2 schema.

Concept: Password Hashing

You never store passwords as plain text. If your database gets hacked, every user's password is exposed — and people reuse passwords across sites.

PHP handles this with two functions:

```
// Registration: turn a password into an unreadable hash
$hashed = password_hash('mypassword', PASSWORD_DEFAULT);
// Result looks like: $2y$10$N.x5Y.a/e5.YSF/dIshg5uEImd...

// Login: check if a submitted password matches the stored hash
if (password_verify('mypassword', $stored_hash)) {
    // correct!
}
```

The hash is one-way — you can't reverse it. Even if someone steals your database, they can't figure out the original passwords.

Example: Users Table

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'staff') DEFAULT 'staff',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Notice the column is `password_hash`, not `password`. That's intentional.

Your Turn

Create the `users` table. Seed one admin account — use this pre-computed hash for password `admin123`:

```
$2y$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi
```

Build `register.php`. The form needs username, email, password, and confirm password fields. The PHP handler should validate everything — empty fields, username length, email format, password match, password length, and

whether the username or email already exists in the database. If everything passes, hash the password and insert. Show clear, specific error messages when validation fails.

Check phpMyAdmin after registering — the `password_hash` column should show `$2y$10$...`, not the actual password.